

## ***1. c. Unrolled List***

### ***–Insert Element***

### ***–Space Complexity Analysis***

#### ***Program:***

```
#define MAX_ELEMENTS 4

// Structure for the Unrolled Linked List Node
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

// Global head pointer
struct Node* head = NULL;

void insertElement(int val) {
    if (head == NULL) {
        head = (struct Node*)malloc(sizeof(struct Node));
        head->numElements = 1;
        head->elements[0] = val;
        head->next = NULL;
        cout << "Inserted " << val << " into a new head node.\n";
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }
}
```

```

else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
}
...

    case 1:
        cout << "Enter integer value to insert: ";
        cin >> value;
        insertElement(value);
        break;

```

## Space Complexity Analysis

### 1. Input Space

#### 1. a. Parametric Input Space: $O(1)$

- The *insertElement* function accepts exactly one parameter: an integer variable named *val*.
- The memory required to hold this single integer value is fixed and does not change regardless of the list's current size.
- Therefore, the parametric input space is strictly constant, resulting in a space complexity of  $O(1)$ .

## **1. b. Full Data Holistic Input Space: $O(N)$**

- *The function operates on a global data structure accessed via the **head** pointer.*
- *The total memory occupied by this entire list scales linearly with the number of existing nodes.*
- *Each node allocates a fixed amount of memory: one integer (**numElements**), an array of 4 integers (**elements[ $MAX\_ELEMENTS$ ]**), and one pointer (**next**).*
- *For a list containing **N** nodes, the overall space occupied in memory by the data structure is  $O(N)$ .*

## **2. Auxiliary Space: $O(1)$**

- *Auxiliary space defines the temporary, extra memory used explicitly to run the algorithm within the function's scope, distinct from the input space.*
- *To perform the traversal, the function declares a single temporary local pointer variable, **struct Node \* temp**.*
- *If the final node's array is full ( $temp \rightarrow numElements < MAX\_ELEMENTS$  evaluates to false), the function allocates a fixed block of memory for exactly one new node using malloc and assigns it to a local pointer **struct Node\* newNode**.*

- *Once the function terminates, the local pointer variables (**temp** and **newNode**) are destroyed.*
- *Because the function uses no recursion (no call stack growth) and creates no dynamic structures that scale with the size of the list, the temporary space utilized is strictly constant, yielding an auxiliary space complexity of  **$O(1)$** .*

***For Academic Answer = Auxiliary Space =  $O(1)$ .***

*"As we often cannot control the size of the input data, but we can control how much extra memory of our algorithm creates to process it."*

***Total Space Complexity or Total memory footprint:***

***= Input Space (Parametric Input Space + Full Data Holistic Input Space)  
+ Auxiliary Space***

***= ( $O(1) + O(N)$ ) +  $O(1)$***

***=  $O(N)$ .***

---

## **Algorithm: UnrolledINSElem(val)**

**((Node \*)malloc(sizeof(Node)) represents the availability of space for the data; let's name it AVAIL.))**

### **1. [CHECK IF LIST IS EMPTY]**

- **If START = NULL, Then:**
  - **SET START := AVAIL** (*Allocating space for the new head node*).
  - **SET numElements[START] := 1**
  - **SET elements[START][0] := val**
  - **SET LINK[START] := NULL**
  - **Write: "Inserted ", val, " into a new head node."**
  - **RETURN**
- **[End of If structure.]**

### **2. [INITIALIZE TEMPORARY POINTER]**

- **SET TEMPPTR := START**

*(Note: The temporary pointer points to the first node to help traverse the loop.)*

### **3. [LOOP TRAVERSAL]**

- **REPEAT WHILE LINK[TEMPPTR] ≠ NULL:**
  - **SET TEMPPTR := LINK[TEMPPTR]**
- **[End of Loop]**

### **4. [CHECK NODE CAPACITY AND INSERT]**

- **If numElements[TEMPPTR] < MAX\_ELEMENTS, Then:**
  - **SET elements[TEMPPTR][numElements[TEMPPTR]] := val**
  - **SET numElements[TEMPPTR] := numElements[TEMPPTR] + 1**
  - **Write: "Inserted ", val, " into the existing last node."**
- **ELSE:**
  - **SET NEWPTR := AVAIL** (*Allocate memory for a newly created node*).
  - **SET numElements[NEWPTR] := 1**
  - **SET elements[NEWPTR][0] := val**
  - **SET LINK[NEWPTR] := NULL**

- $SET\ LINK[TEMPPTR] :=$   
 $NEWPTR \left( \begin{array}{l} \text{Linking the new node with} \\ \text{the previous node after which} \\ \text{it is to be placed} \end{array} \right).$
  
- Write: "Node full.Inserted ",val," into a newly created node."
  
- [End of If structure.]

## 5. END OF ALGORITHM

---